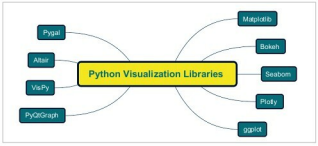

Figure 1: Python visualisation libraries

```
axis_label='y')

#4.  add a line renderer with legend and line thickness
p.line(x, y, legend="Age", line_width=3)

#5. show the results
show(p)
```

The output of the above code is shown in Figure 2.

The major steps involved in building a plot with Bokeh are listed below:

- Load the data required for plotting in variables.
- Specify the name of the output HTML file. The visualisation built with Bokeh will be saved as an HTML file and the output loaded in the browser.
- Use the *figure()* function to build a plot with options.
- Specific graphs can be constructed using a renderer. In the earlier-mentioned example, the renderer used is *Figure.line.*

The final step is to call the *show()* or *save()* function.

The concepts involved in building visualisations using Bokeh are:

- Plot
- Glyphs
- Guides and annotations

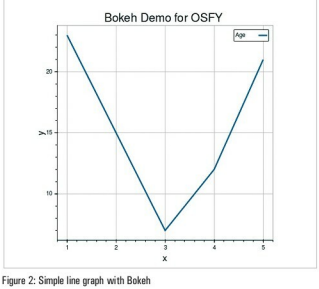

Figure 2: Simple line graph with Bokeh

- Ranges
- Resources

Detailed descriptions about these concepts are provided in the official documentation (*http://bokeh.pydata.org/en/latest/docs/reference.html#refguide*).

Bokeh facilitates linking various factors of different plots, which is referred to as linked panning. Here, some components are shared across multiple plots. Changing the range of one plot will update other plots as well. The sample code is given below and its output is shown in Figure 3.

```
import numpy as np
from bokeh.layouts import gridplot
from bokeh.plotting import figure, output_file, show
# prepare some data
N = 100
x = np.linspace(0, 4*np.pi, N)
y0 = np.sin(x)
y1 = np.cos(x)
y2 = np.sin(x) + np.cos(x)
# output to static HTML file
output_file("linked_panning.html")
# create a new plot
s1 = figure(width=250, plot_height=250, title=None)
s1.circle(x, y0, size=10, color="blue", alpha=0.5)
# NEW: create a new plot and share both ranges
s2 = figure(width=250, height=250, x_range=s1.x_range, y_
range=s1.y_range, title=None)
s2.triangle(x, y1, size=10, color="firebrick", alpha=0.5)
# NEW: create a new plot and share only one range
s3 = figure(width=250, height=250, x_range=s1.x_range,
title=None)
s3.square(x, y2, size=10, color="green", alpha=0.5)
# NEW: put the subplots in a gridplot
p = gridplot([[s1, s2, s3]], toolbar_location=None)
# show the results
show(p)
```

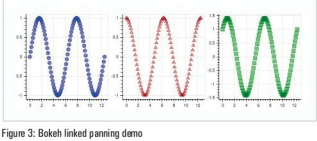

Figure 3: Bokeh linked panning demo

Altair

Altair is based on the declarative statistical visualisation approach available for Python. It is based on the high-level Vega-Lite visualisation grammar that provides JSON syntax for the production of visualisations *(https://vega.github.io/vega-lite/)*.